

1. Import libraries

```
In [5]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

2. Load Titanic data

```
In [6]: kashti = sns.load_dataset('titanic')
```

```
In [7]: kashti.head()
```

```
Out[7]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

```
In [8]: kashti.tail()
```

```
Out[8]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult
886	0	2	male	27.0	0	0	13.00	S	Second	man	
887	1	1	female	19.0	0	0	30.00	S	First	woman	
888	0	3	female	NaN	1	2	23.45	S	Third	woman	
889	1	1	male	26.0	0	0	30.00	C	First	man	
890	0	3	male	32.0	0	0	7.75	Q	Third	man	

```
In [9]: kashti.shape
```

```
Out[9]: (891, 15)
```

```
In [10]: kashti.columns
```

```
Out[10]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
               'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
               'alive', 'alone'],
              dtype='object')
```

```
In [11]: kashti.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   survived        891 non-null    int64
1   pclass          891 non-null    int64
2   sex             891 non-null    object
3   age             714 non-null    float64
4   sibsp          891 non-null    int64
5   parch          891 non-null    int64
6   fare           891 non-null    float64
7   embarked        889 non-null    object
8   class          891 non-null    category
9   who            891 non-null    object
10  adult_male      891 non-null    bool
11  deck           203 non-null    category
12  embark_town     889 non-null    object
13  alive          891 non-null    object
14  alone          891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

```
In [12]: kashti[kashti["age"] < 5].head()
```

```
Out[12]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_n
7	0	3	male	2.0	3	1	21.0750	S	Third	child	F
10	1	3	female	4.0	1	1	16.7000	S	Third	child	F
16	0	3	male	2.0	4	1	29.1250	Q	Third	child	F
43	1	2	female	3.0	1	2	41.5792	C	Second	child	F
63	0	3	male	4.0	3	2	27.9000	S	Third	child	F

```
In [13]: kashti[kashti["age"] < 1] # anyone under age 1 year traveled free?
```

```
Out[13]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_n
78	1	2	male	0.83	0	2	29.0000	S	Second	child	
305	1	1	male	0.92	1	2	151.5500	S	First	child	
469	1	3	female	0.75	2	1	19.2583	C	Third	child	
644	1	3	female	0.75	2	1	19.2583	C	Third	child	
755	1	2	male	0.67	1	1	14.5000	S	Second	child	
803	1	3	male	0.42	0	1	8.5167	C	Third	child	
831	1	2	male	0.83	1	1	18.7500	S	Second	child	

```
In [14]: kashti[kashti["fare"] < 5].head()
```

```
Out[14]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
179	0	3	male	36.0	0	0	0.0	S	Third	man	True
263	0	1	male	40.0	0	0	0.0	S	First	man	True
271	1	3	male	25.0	0	0	0.0	S	Third	man	True
277	0	2	male	NaN	0	0	0.0	S	Second	man	True
302	0	3	male	19.0	0	0	0.0	S	Third	man	True

3. Basic understanding

```
In [15]: kashti.describe()
```

```
Out[15]:
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [16]: kashti.describe(include="object")
```

```
Out[16]:
```

	sex	embarked	who	embark_town	alive
count	891	889	891	889	891
unique	2	3	3	3	2
top	male	S	man	Southampton	no
freq	577	644	537	644	549

```
In [17]: kashti.dtypes
```

```
Out[17]: survived      int64
         pclass        int64
         sex           object
         age           float64
         sibsp         int64
         parch         int64
         fare          float64
         embarked      object
         class         category
         who           object
         adult_male    bool
         deck          category
         embark_town   object
         alive         object
         alone         bool
         dtype: object
```

```
In [18]: kashti.nunique()
```

```
Out[18]: survived      2
         pclass        3
         sex           2
         age           88
         sibsp         7
         parch         7
         fare          248
         embarked      3
         class         3
         who           3
         adult_male    2
         deck          7
         embark_town   3
         alive         2
         alone         2
         dtype: int64
```

4. Missing values

```
In [19]: kashti.isnull().sum()
```

```
Out[19]: survived      0
         pclass        0
         sex           0
         age          177
         sibsp         0
         parch         0
         fare          0
         embarked      2
         class         0
         who           0
         adult_male     0
         deck          688
         embark_town    2
         alive          0
         alone          0
         dtype: int64
```

```
In [20]: missing = kashti.isnull().sum().sort_values(ascending=False)

missing_percent = (kashti.isnull().sum() / len(kashti) * 100).sort_values(
    ascending=False
)

missing_Kashti = pd.DataFrame({
    "Missing": missing,
    "Percentage": missing_percent
})

missing_Kashti
```

Out[20]:

	Missing	Percentage
deck	688	77.216611
age	177	19.865320
embarked	2	0.224467
embark_town	2	0.224467
sex	0	0.000000
pclass	0	0.000000
survived	0	0.000000
fare	0	0.000000
parch	0	0.000000
sibsp	0	0.000000
class	0	0.000000
adult_male	0	0.000000
who	0	0.000000
alive	0	0.000000
alone	0	0.000000

5. Duplicate records

```
In [21]: kashti.duplicated().sum()
```

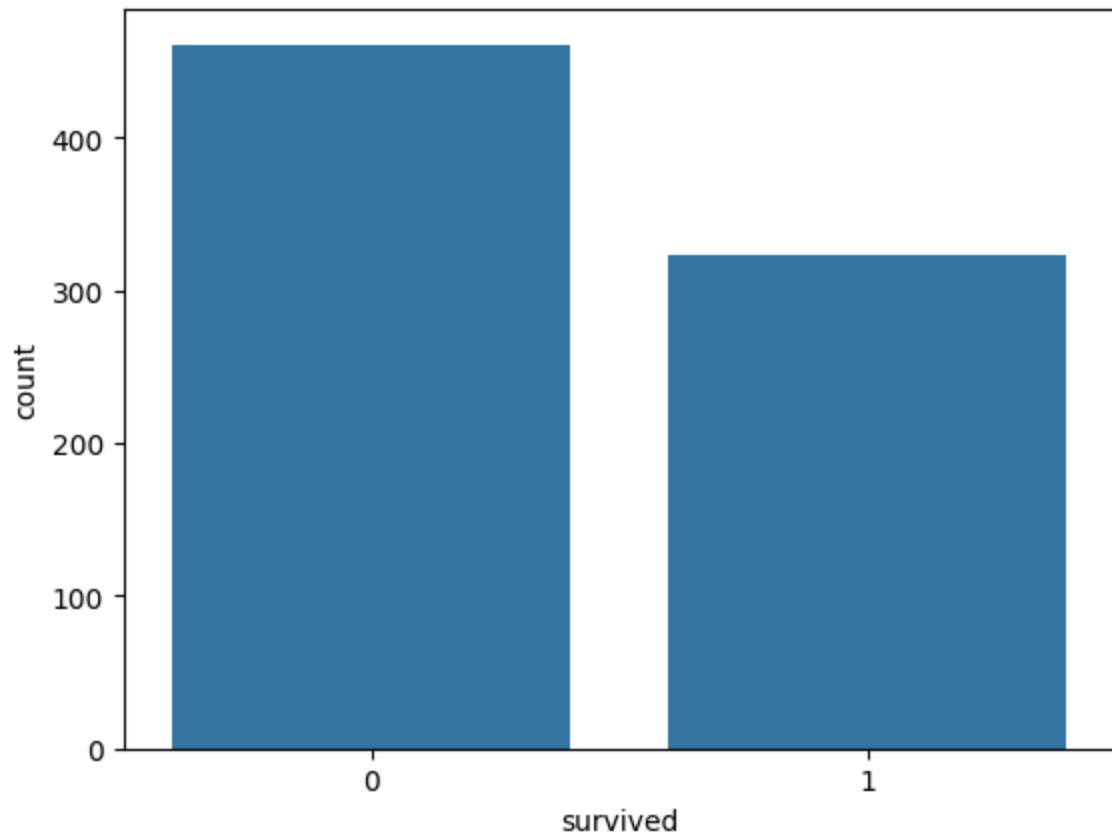
Out[21]: np.int64(107)

```
In [22]: kashti = kashti.drop_duplicates()
```

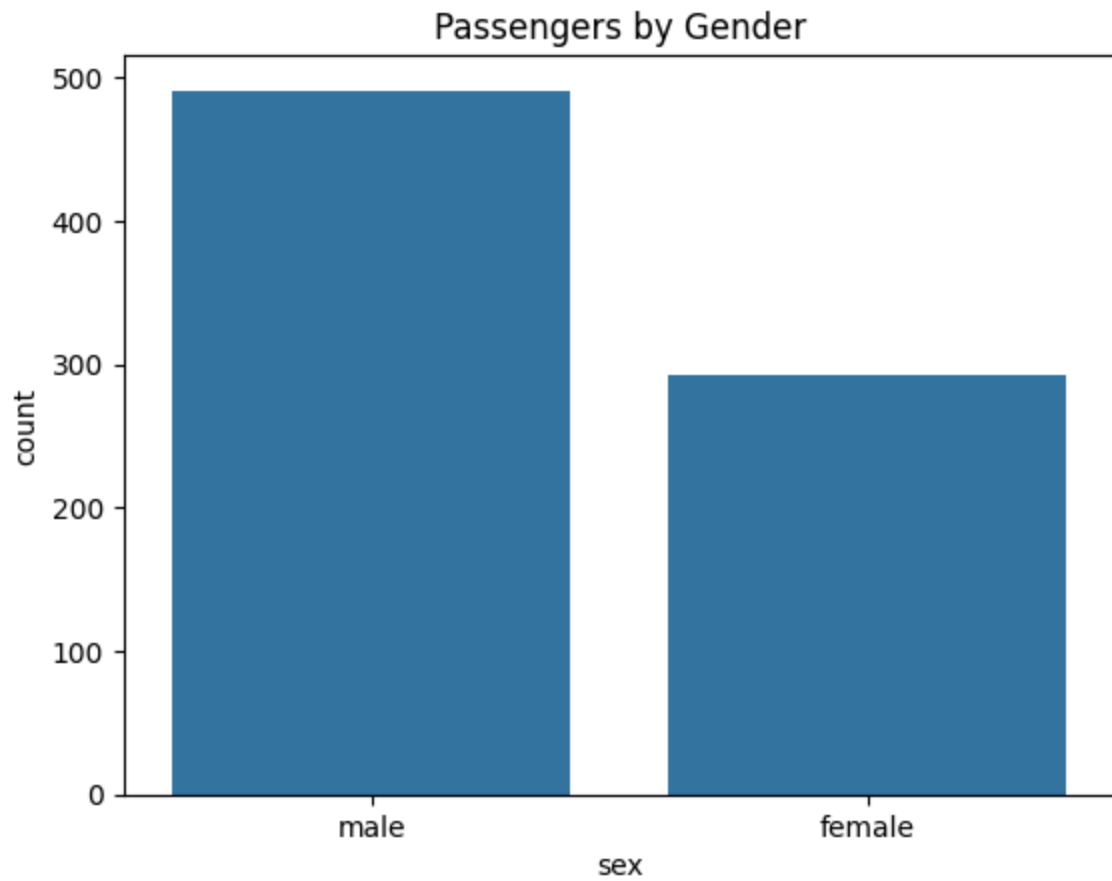
6. Univariate Analysis

```
In [70]: import pandas as pd
import seaborn as sns

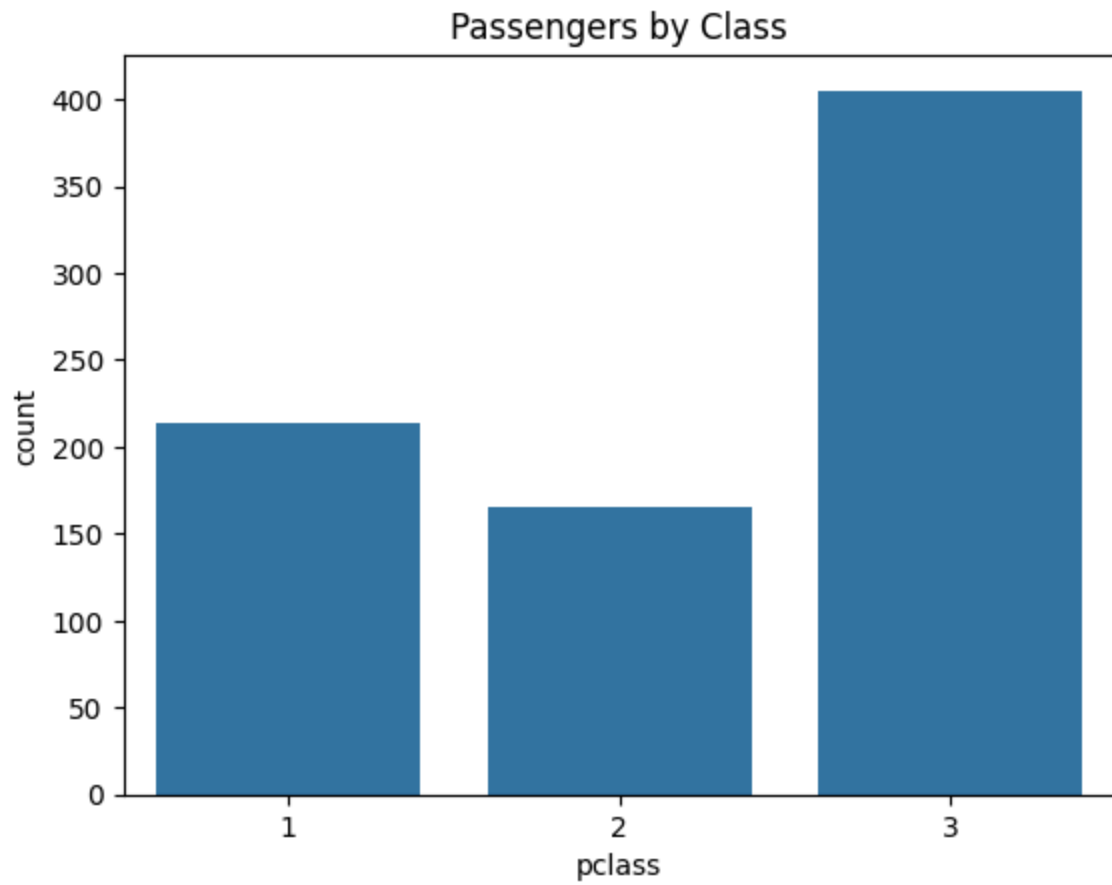
#Survival
sns.countplot(data=kashti, x="survived")
plt.show()
```



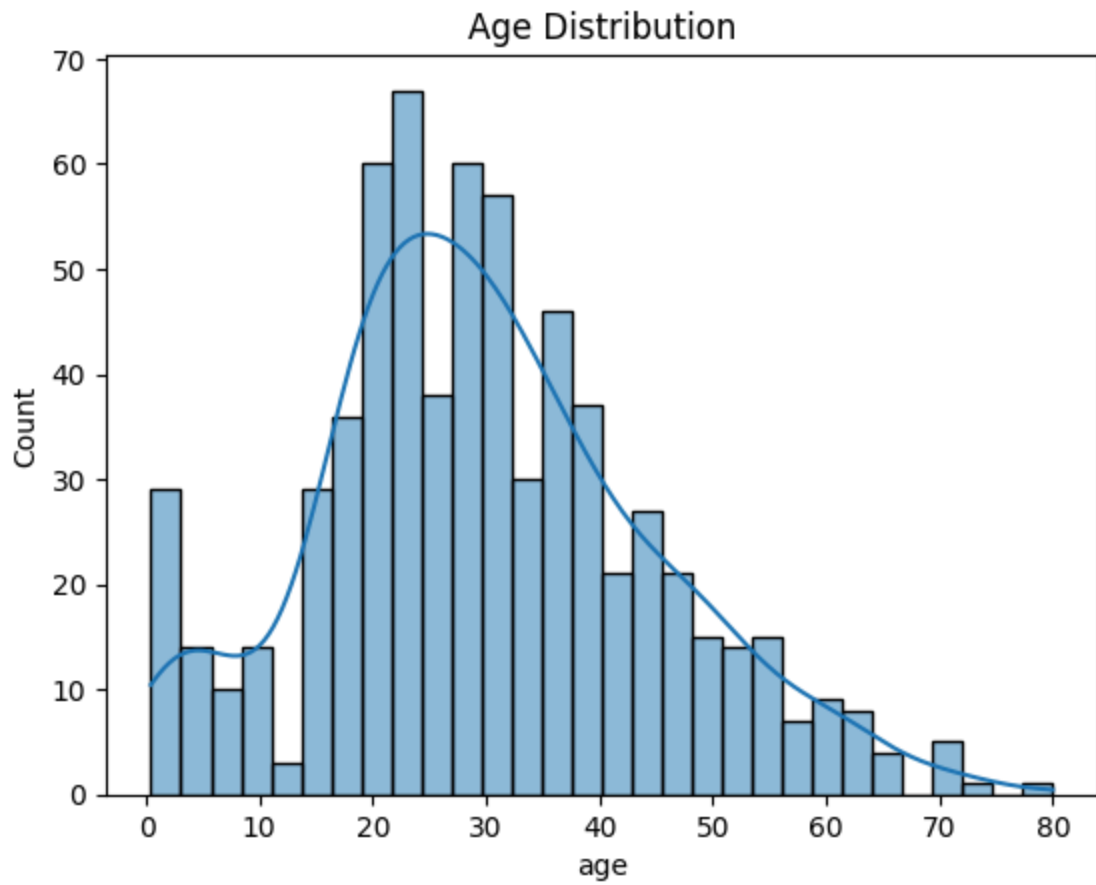
```
In [24]: #Gender
sns.countplot(data=kashti, x="sex")
plt.title("Passengers by Gender")
plt.show()
```



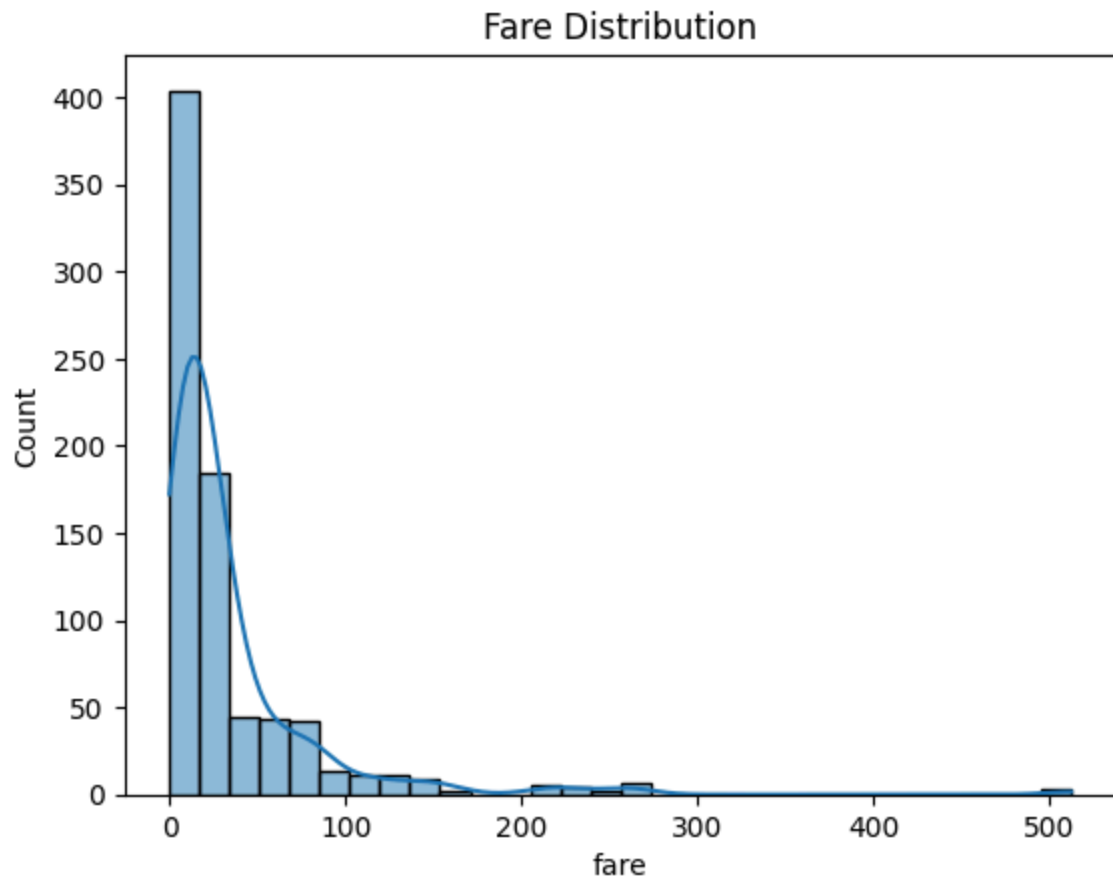
```
In [25]: #Passenger class
sns.countplot(data=kashti, x="pclass")
plt.title("Passengers by Class")
plt.show()
```

```
In [26]: #Age distribution
sns.histplot(data=kashti, x="age", bins=30, kde=True)
plt.title("Age Distribution")
plt.show()
```

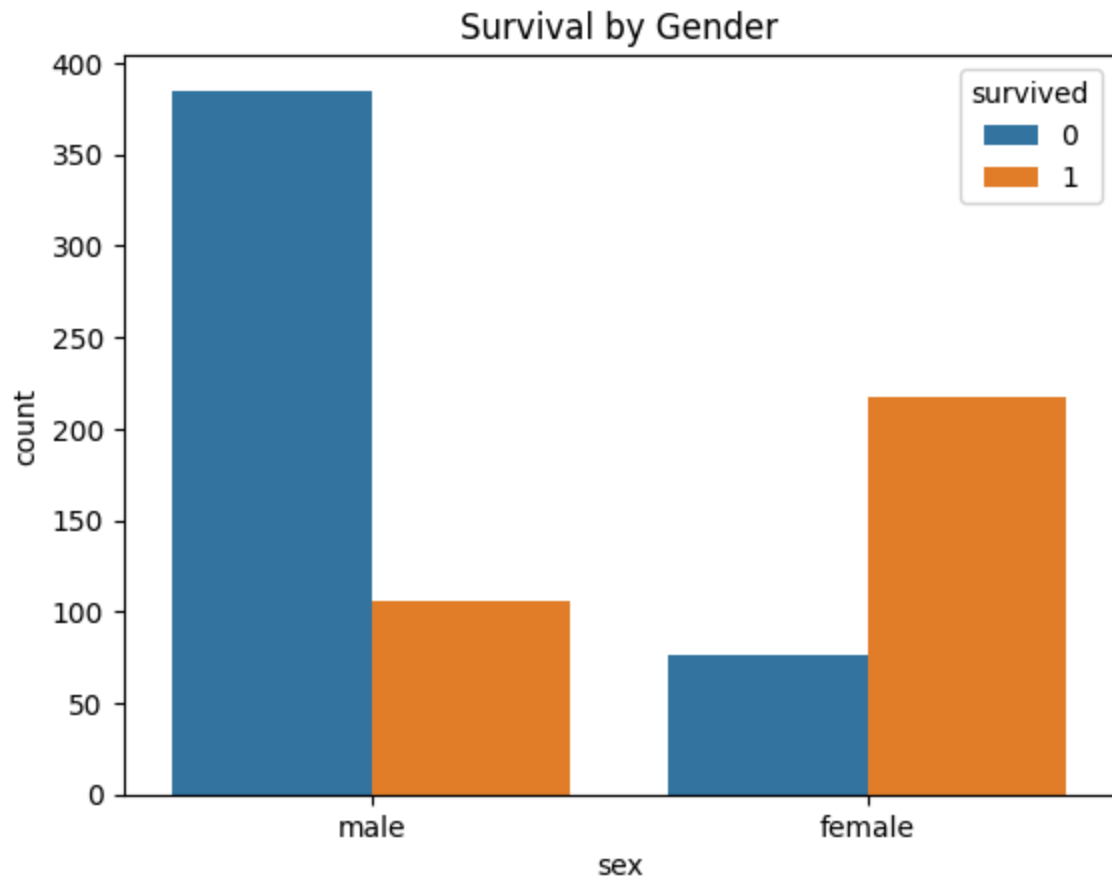


```
In [27]: #Fare distribution
sns.histplot(data=kashti, x="fare", bins=30, kde=True)
plt.title("Fare Distribution")
plt.show()
```

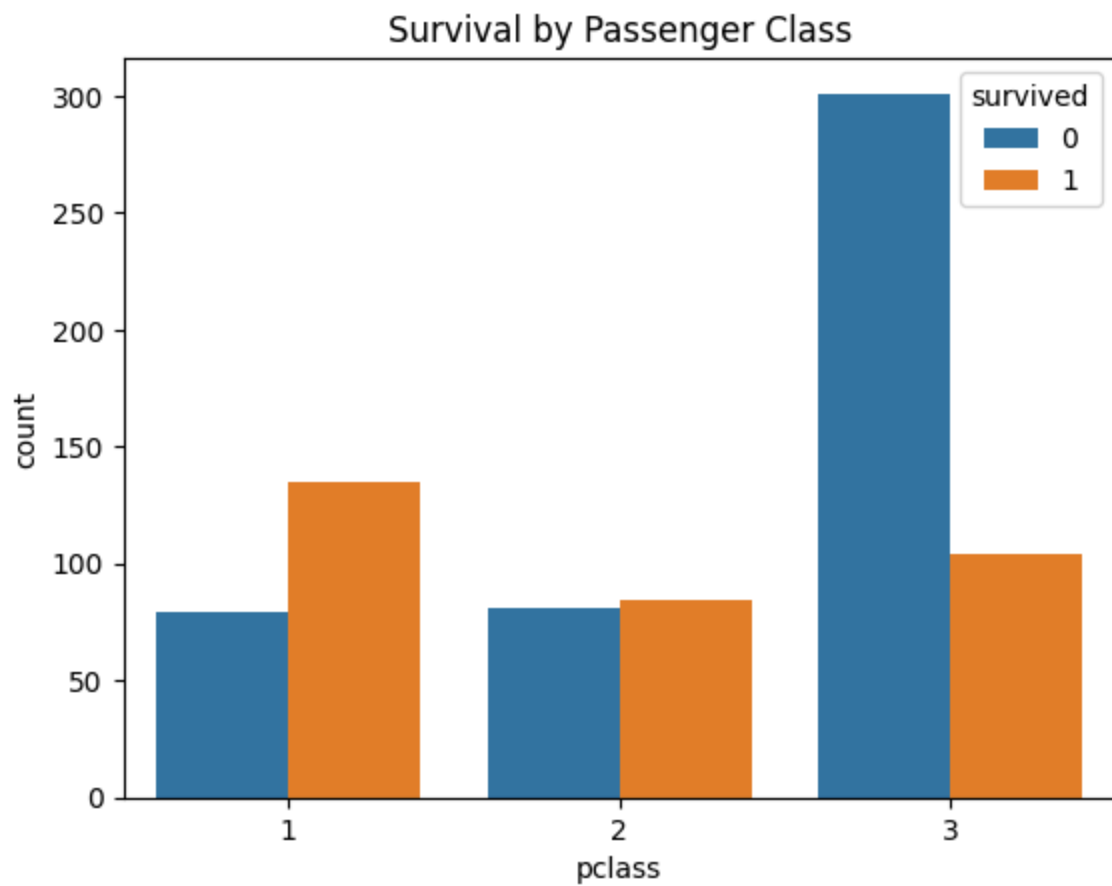


7. Bivariate Analysis

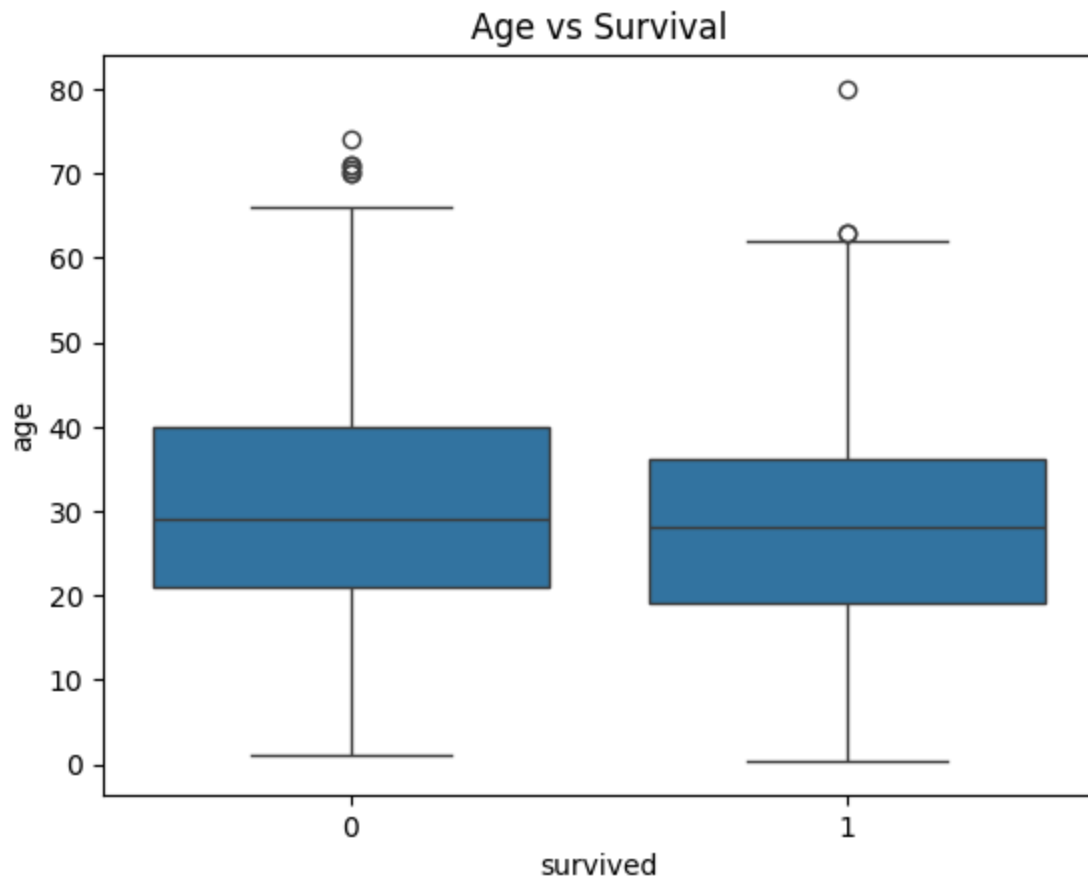
```
In [28]: #Gender vs Survival
sns.countplot(data=kashti, x="sex", hue="survived")
plt.title("Survival by Gender")
plt.show()
```



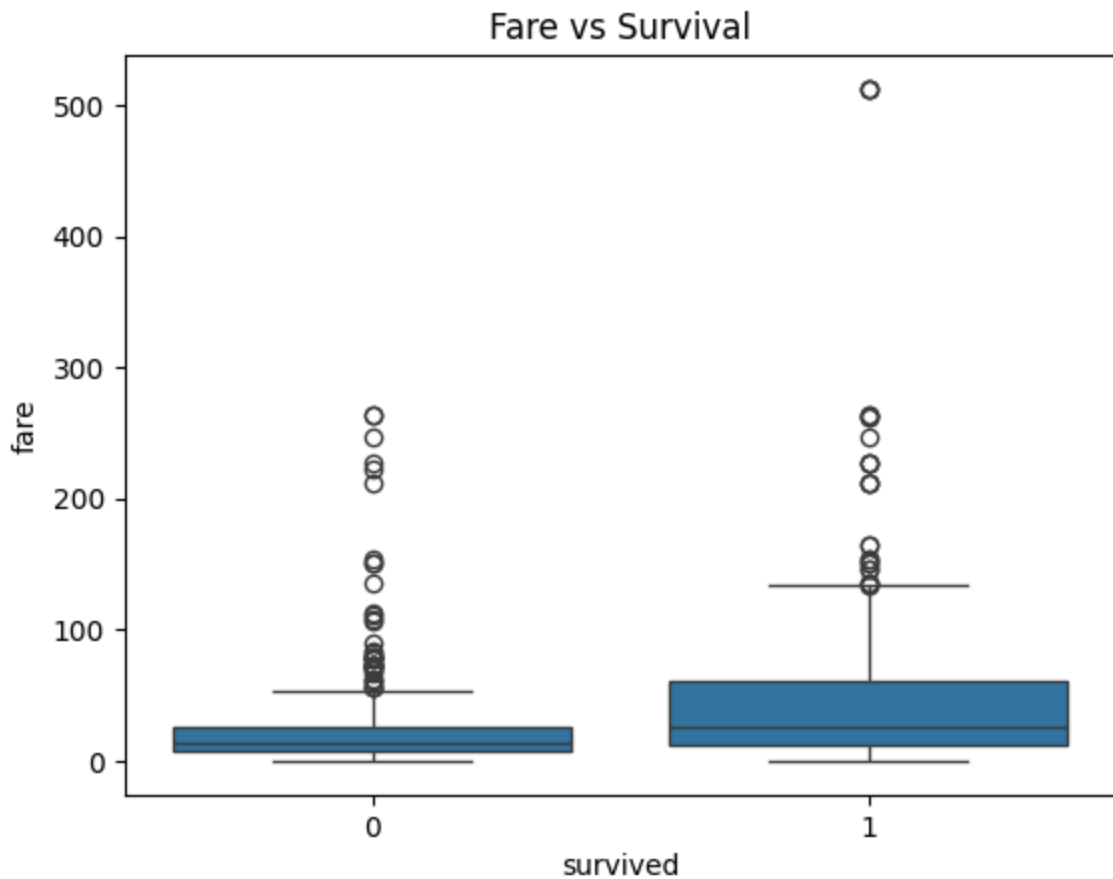
```
In [29]: # Passenger class vs Survival
sns.countplot(data=kashti, x="pclass", hue="survived")
plt.title("Survival by Passenger Class")
plt.show()
```



```
In [30]: ## Age vs Survival
sns.boxplot(data=kashti, x="survived", y="age")
plt.title("Age vs Survival")
plt.show()
```



```
In [31]: #Fare vs Survival
sns.boxplot(data=kashti, x="survived", y="fare")
plt.title("Fare vs Survival")
plt.show()
```



8. Calculate survival rates

```
In [71]: #Overall survival rate  
kashti["survived"].mean()
```

```
Out[71]: np.float64(0.4119897959183674)
```

```
In [72]: #Survival by gender  
kashti.groupby("sex")["survived"].mean()
```

```
Out[72]: sex  
female    0.740614  
male      0.215886  
Name: survived, dtype: float64
```

```
In [73]: #Survival by class  
kashti.groupby("pclass")["survived"].mean()
```

```
Out[73]: pclass  
1      0.630841  
2      0.509091  
3      0.256790  
Name: survived, dtype: float64
```

```
In [74]: #Survival by gender and class  
kashti.groupby(["sex", "pclass"])["survived"].mean()
```

```
Out[74]: sex      pclass
female 1      0.967742
        2      0.917808
        3      0.472441
male    1      0.371901
        2      0.184783
        3      0.158273
Name: survived, dtype: float64
```

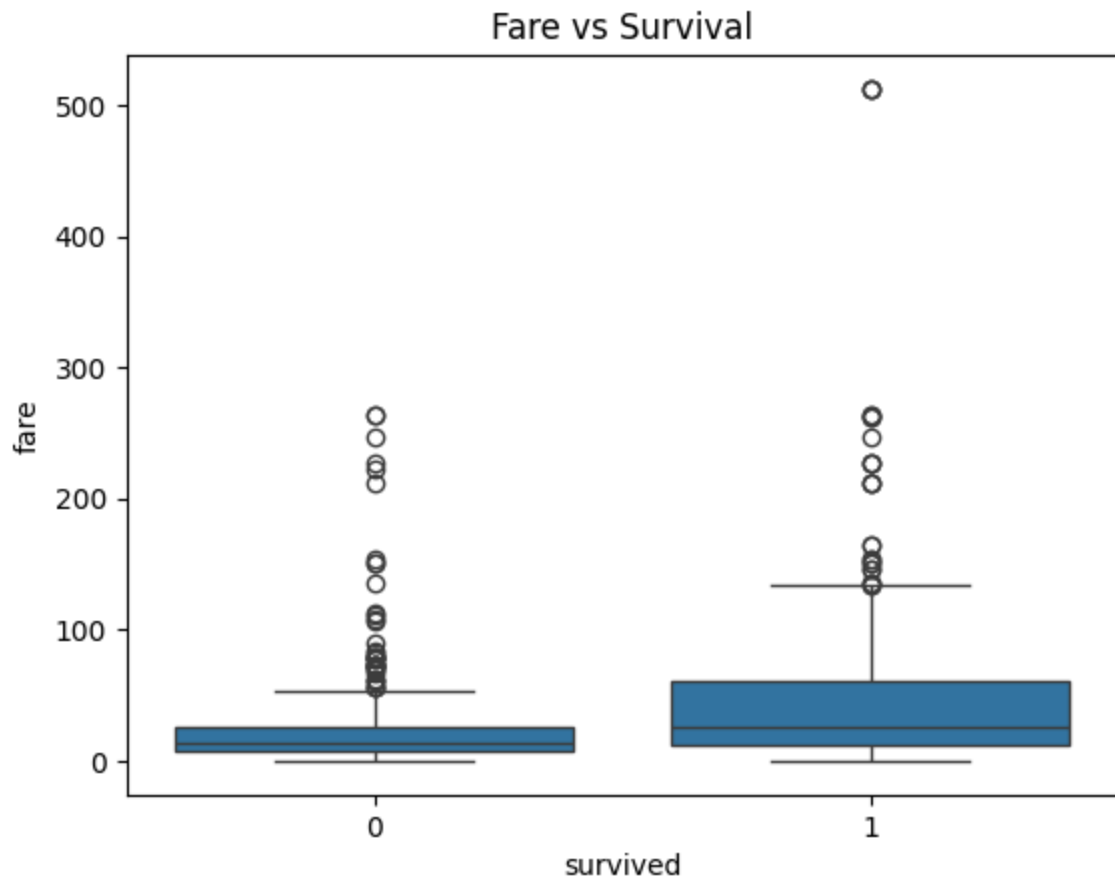
```
In [75]: survival_table = pd.pivot_table(
    kashti,
    values="survived",
    index="sex",
    columns="pclass",
    aggfunc="mean"
)

survival_table
```

```
Out[75]:
```

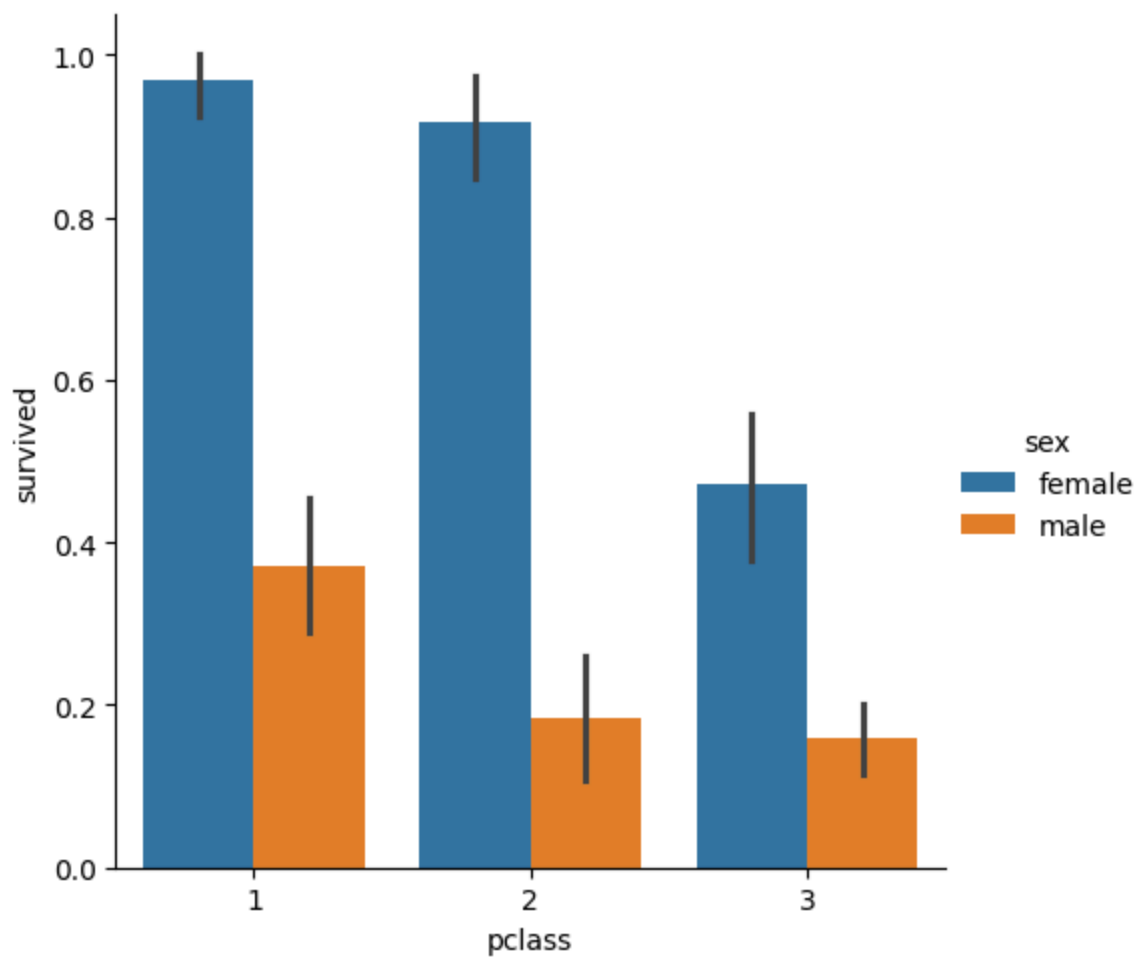
	pclass	1	2	3
sex				
female		0.967742	0.917808	0.472441
male		0.371901	0.184783	0.158273

```
In [76]: # Fare vs Survival
sns.boxplot(data=kashti, x="survived", y="fare")
plt.title("Fare vs Survival")
plt.show()
```

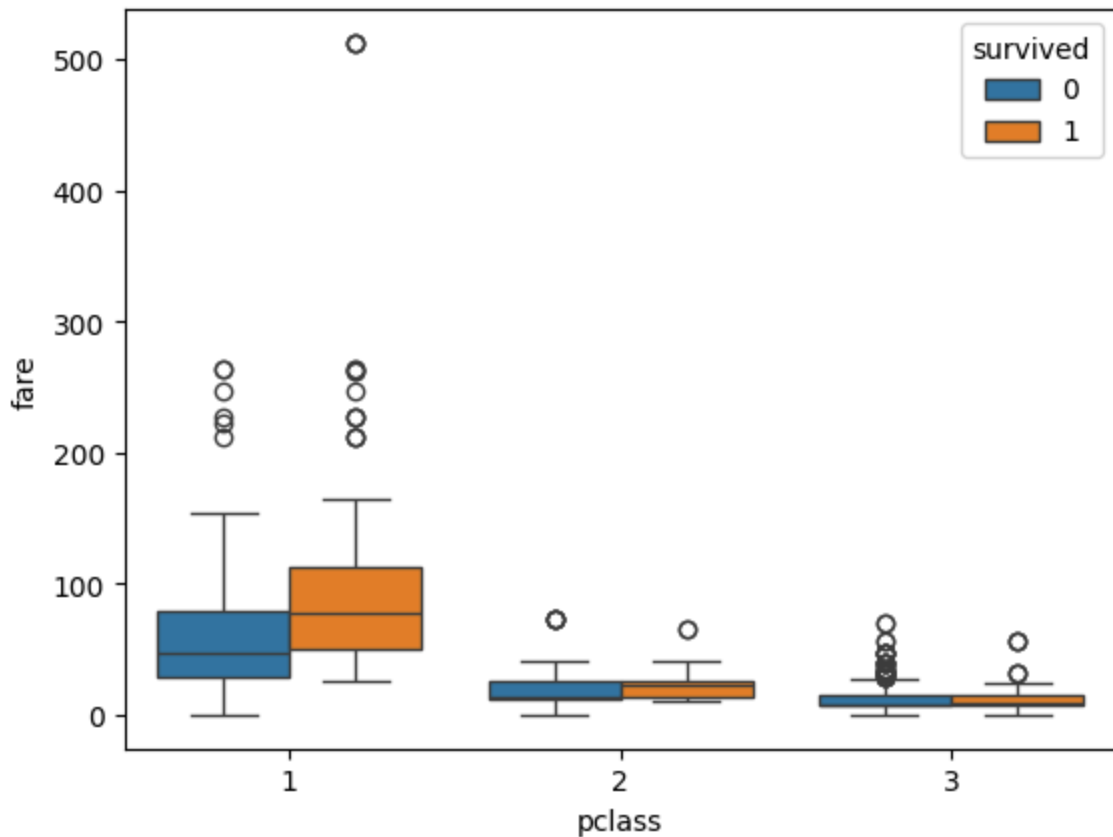



9. Multivariate Analysis

```
In [77]: sns.catplot(  
    data=kashti,  
    x="pclass",  
    y="survived",  
    hue="sex",  
    kind="bar"  
)  
  
plt.show()
```



```
In [78]: sns.boxplot(  
    data=kashti,  
    x="pclass",  
    y="fare",  
    hue="survived"  
)  
  
plt.show()
```



10. Correlation analysis

```
In [ ]: numeric_kashti = kashti.select_dtypes(include=np.number)
numeric_kashti.corr()
```

	survived	pclass	age	sibsp	parch	fare	FamilySize	
survived	1.000000	-0.332658	-0.079184	-0.036589	0.070307	0.246769	0.014996	-
pclass	-0.332658	1.000000	-0.342986	0.088014	0.040296	-0.549216	0.079425	-
age	-0.079184	-0.342986	1.000000	-0.279431	-0.183777	0.091315	-0.282962	-
sibsp	-0.036589	0.088014	-0.279431	1.000000	0.381433	0.135147	0.860279	-
parch	0.070307	0.040296	-0.183777	0.381433	1.000000	0.191942	0.799417	-
fare	0.246769	-0.549216	0.091315	0.135147	0.191942	1.000000	0.193694	-
FamilySize	0.014996	0.079425	-0.282962	0.860279	0.799417	0.193694	1.000000	-
Alone	-0.176595	0.109545	0.191706	-0.609502	-0.571087	-0.246619	-0.711088	-
CabinKnown	0.306812	-0.720880	0.235168	-0.046274	0.017453	0.470893	-0.020448	-

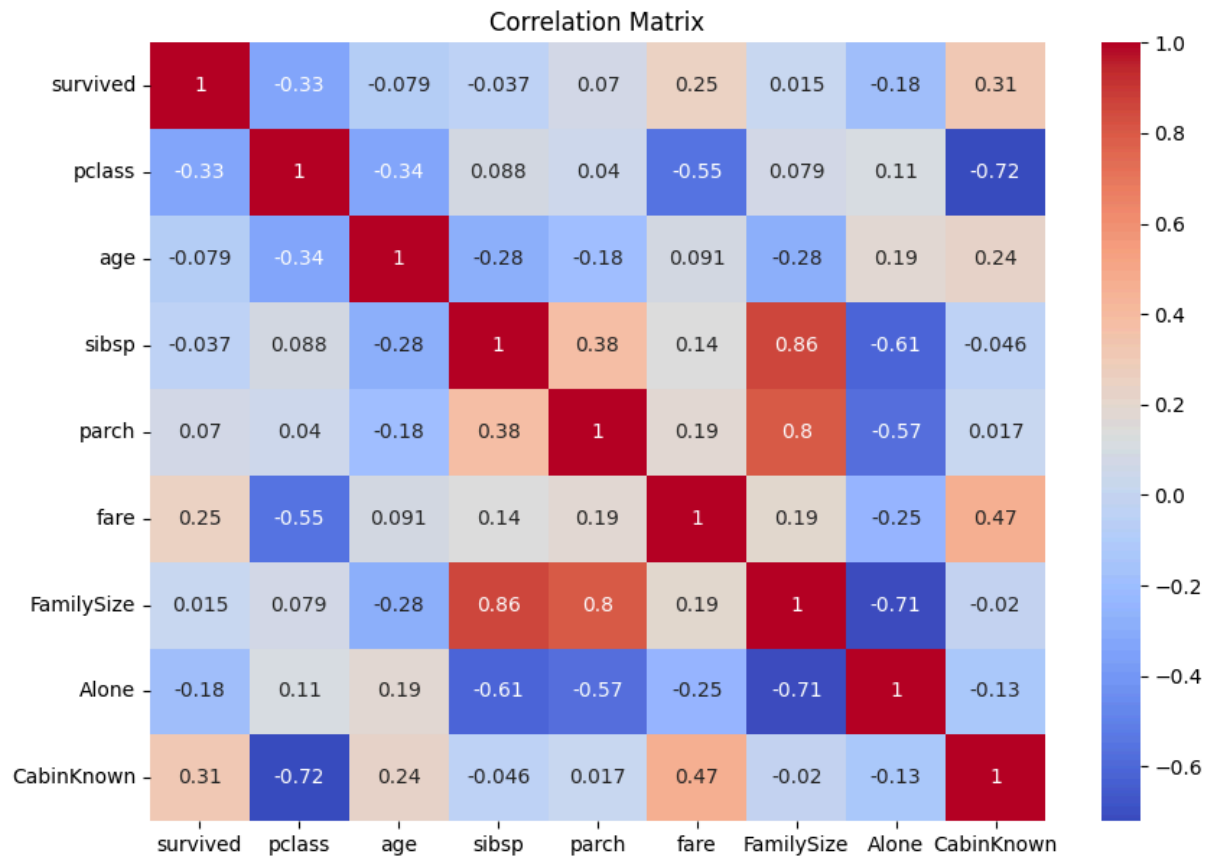
```
In [ ]: plt.figure(figsize=(10, 7))
sns.heatmap(
    numeric_kashti.corr(),
    annot=True,
```

```

    cmap="coolwarm"
)

plt.title("Correlation Matrix")
plt.show()

```

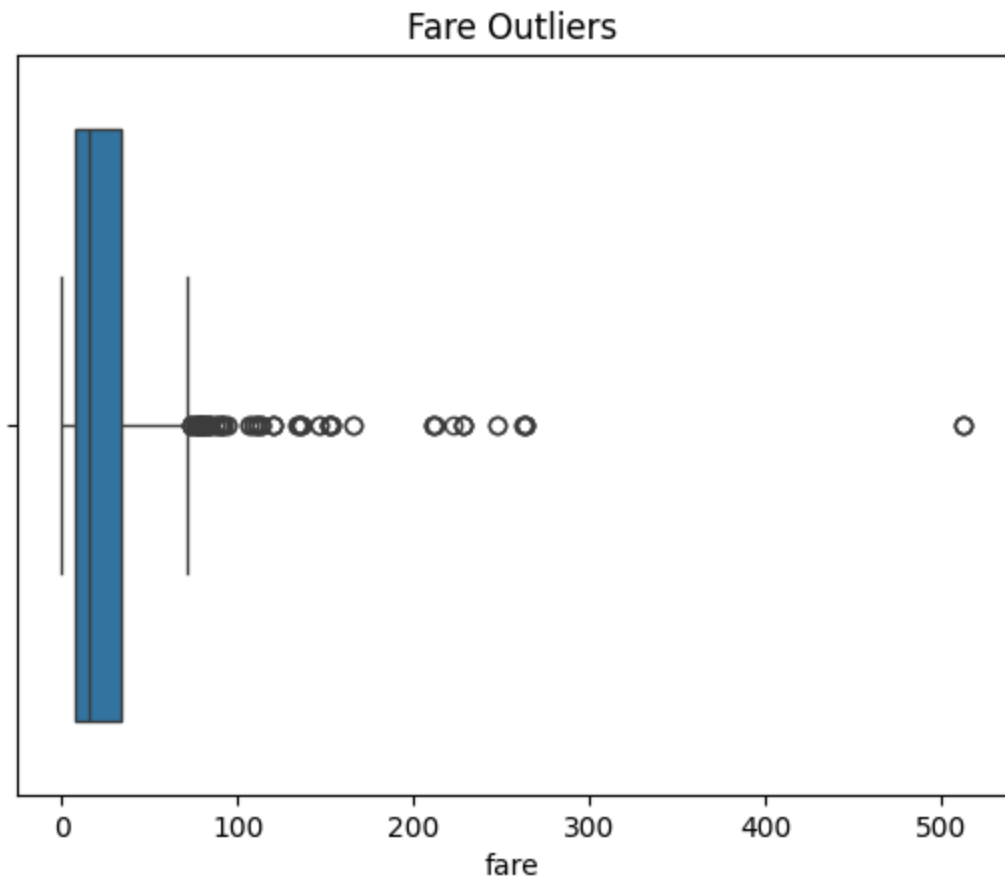


11. Outlier analysis

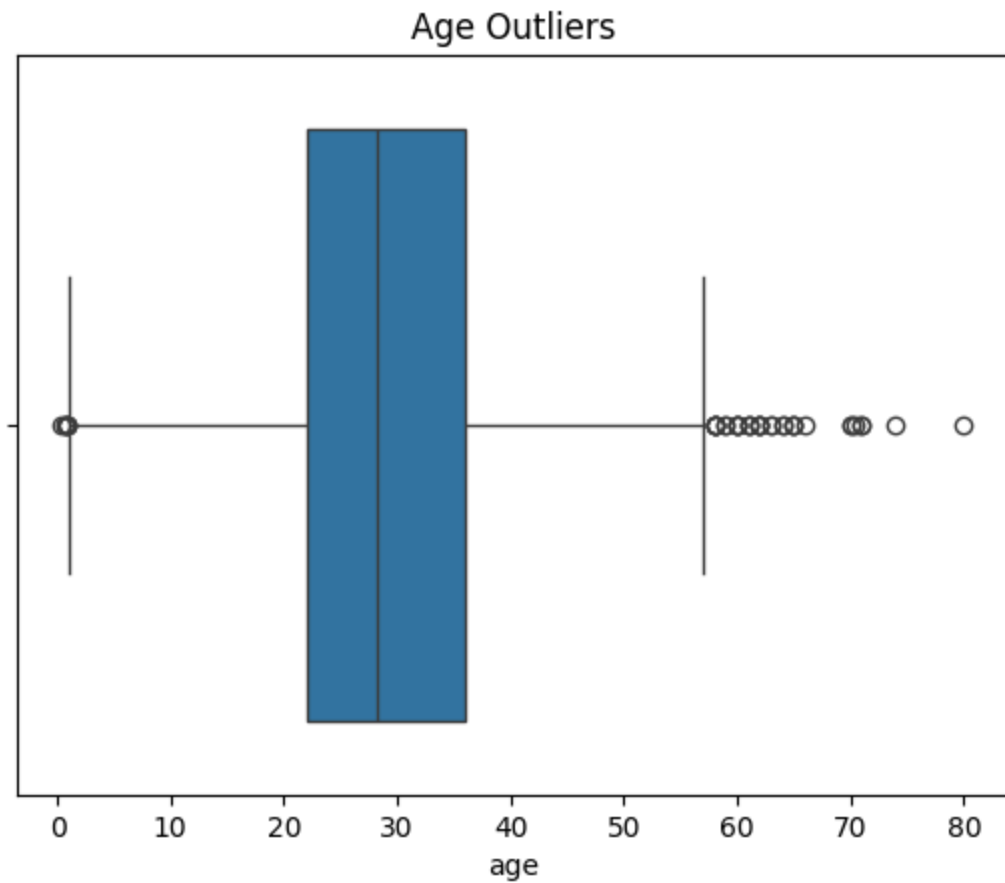
```

In [110... #Fare:
sns.boxplot(data=kashti, x="fare")
plt.title("Fare Outliers")
plt.show()

```



```
In [82]: #Age:
sns.boxplot(data=kashti, x="age")
plt.title("Age Outliers")
plt.show()
```



```
In [83]: #Calculate IQR:
Q1 = kashti["fare"].quantile(0.25)
Q3 = kashti["fare"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = kashti[
    (kashti["fare"] < lower) |
    (kashti["fare"] > upper)
]

outliers
```

Out[83]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	...
27	0	1	male	19.00	3	2	263.0000	S	First	man	...
31	1	1	female	28.25	1	0	146.5208	C	First	woman	...
34	0	1	male	28.00	1	0	82.1708	C	First	man	...
52	1	1	female	49.00	1	0	76.7292	C	First	woman	...
61	1	1	female	38.00	0	0	80.0000	S	First	woman	...
...	...	...	...	...	...	...	...	...	...	...	...
829	1	1	female	62.00	0	0	80.0000	S	First	woman	...
835	1	1	female	39.00	1	1	83.1583	C	First	woman	...
849	1	1	female	28.25	1	0	89.1042	C	First	woman	...
856	1	1	female	45.00	1	1	164.8667	S	First	woman	...
879	1	1	female	56.00	0	1	83.1583	C	First	woman	...

102 rows × 21 columns

12. Feature relationships

In [84]: *#Did women survive more than men?*

```
pd.crosstab(  
    kashti["sex"],  
    kashti["survived"],  
    normalize="index"  
)
```

Out[84]:

survived	0	1
----------	---	---

sex		
female	0.259386	0.740614
male	0.784114	0.215886

In [85]: *#Did first-class passengers survive more?*

```
pd.crosstab(  
    kashti["pclass"],  
    kashti["survived"],  
    normalize="index"  
)
```

```
Out[85]: survived      0      1
```

pclass		
1	0.369159	0.630841
2	0.490909	0.509091
3	0.743210	0.256790

```
In [86]: # Did children have better survival?
kashti["Age_Group"] = pd.cut(
    kashti["age"],
    bins=[0, 12, 18, 35, 60, 100],
    labels=["Child", "Teenager", "Young Adult", "Adult", "Senior"]
)
```

```
In [87]: kashti.groupby("Age_Group", observed=True)["survived"].mean()
```

```
Out[87]: Age_Group
Child      0.573529
Teenager   0.447761
Young Adult 0.395402
Adult      0.401042
Senior     0.227273
Name: survived, dtype: float64
```

13. Feature engineering

```
In [88]: #Create family size:
kashti["FamilySize"] = kashti["sibsp"] + kashti["parch"] + 1
```

```
In [89]: # Alone passengers
kashti["Alone"] = (kashti["FamilySize"] == 1).astype(int)
```

```
In [90]: kashti.columns
```

```
Out[90]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
               'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
               'alive', 'alone', 'Age_Group', 'FamilySize', 'Alone', 'Name', 'Title',
               'CabinKnown'],
              dtype='object')
```

```
In [91]: kashti["survived"].value_counts()
```

```
Out[91]: survived
0      461
1      323
Name: count, dtype: int64
```

```
In [92]: kashti.groupby("sex")["survived"].mean()
```



```
Out[92]: sex
female    0.740614
male      0.215886
Name: survived, dtype: float64
```

```
In [93]: kashti.groupby("pclass")["survived"].mean()
```

```
Out[93]: pclass
1    0.630841
2    0.509091
3    0.256790
Name: survived, dtype: float64
```

```
In [94]: kashti.groupby("who")["survived"].mean()
```

```
Out[94]: who
child    0.585366
man      0.188470
woman    0.756972
Name: survived, dtype: float64
```

Add a Name column for all 891 rows

```
In [111... def create_name(row):
    if row["age"] < 14:
        title = "Master"
    elif row["sex"] == "female":
        title = "Miss"
    else:
        title = "Mr"

    return f"{title}"
```

```
In [96]: kashti["Name"] = kashti.apply(create_name, axis=1)
```

```
In [97]: kashti[["Name", "sex", "age"]].head(10)
```

```
Out[97]:
```

	Name	sex	age
0	Mr	male	22.00
1	Miss	female	38.00
2	Miss	female	26.00
3	Miss	female	35.00
4	Mr	male	35.00
5	Mr	male	28.25
6	Mr	male	54.00
7	Master	male	2.00
8	Miss	female	27.00
9	Miss	female	14.00

```
In [98]: kashti["Title"] = kashti["Name"].str.extract(
          r",\s*([^.]+)\.",
          )[0]
```

```
In [99]: kashti[["Name", "Title"]].head(10)
```

```
Out[99]:
```

	Name	Title
0	Mr	NaN
1	Miss	NaN
2	Miss	NaN
3	Miss	NaN
4	Mr	NaN
5	Mr	NaN
6	Mr	NaN
7	Master	NaN
8	Miss	NaN
9	Miss	NaN

```
In [100... kashti["Title"].value_counts()
```

```
Out[100... Series([], Name: count, dtype: int64)
```

14. Missing-value treatment

```
In [101... kashti["age"] = kashti["age"].fillna(kashti["age"].median())
```

```
In [102... kashti["embarked"] = kashti["embarked"].fillna(  
    kashti["embarked"].mode()[0]  
)
```

```
In [103... kashti.columns
```

```
Out[103... Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',  
      'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',  
      'alive', 'alone', 'Age_Group', 'FamilySize', 'Alone', 'Name', 'Title',  
      'CabinKnown'],  
      dtype='object')
```

```
In [104... kashti["CabinKnown"] = kashti["deck"].notna().astype(int)
```

```
In [105... kashti[["deck", "CabinKnown"]].head(10)
```

```
Out[105... 

|   | deck | CabinKnown |
|---|------|------------|
| 0 | NaN  | 0          |
| 1 | C    | 1          |
| 2 | NaN  | 0          |
| 3 | C    | 1          |
| 4 | NaN  | 0          |
| 5 | NaN  | 0          |
| 6 | E    | 1          |
| 7 | NaN  | 0          |
| 8 | NaN  | 0          |
| 9 | NaN  | 0          |

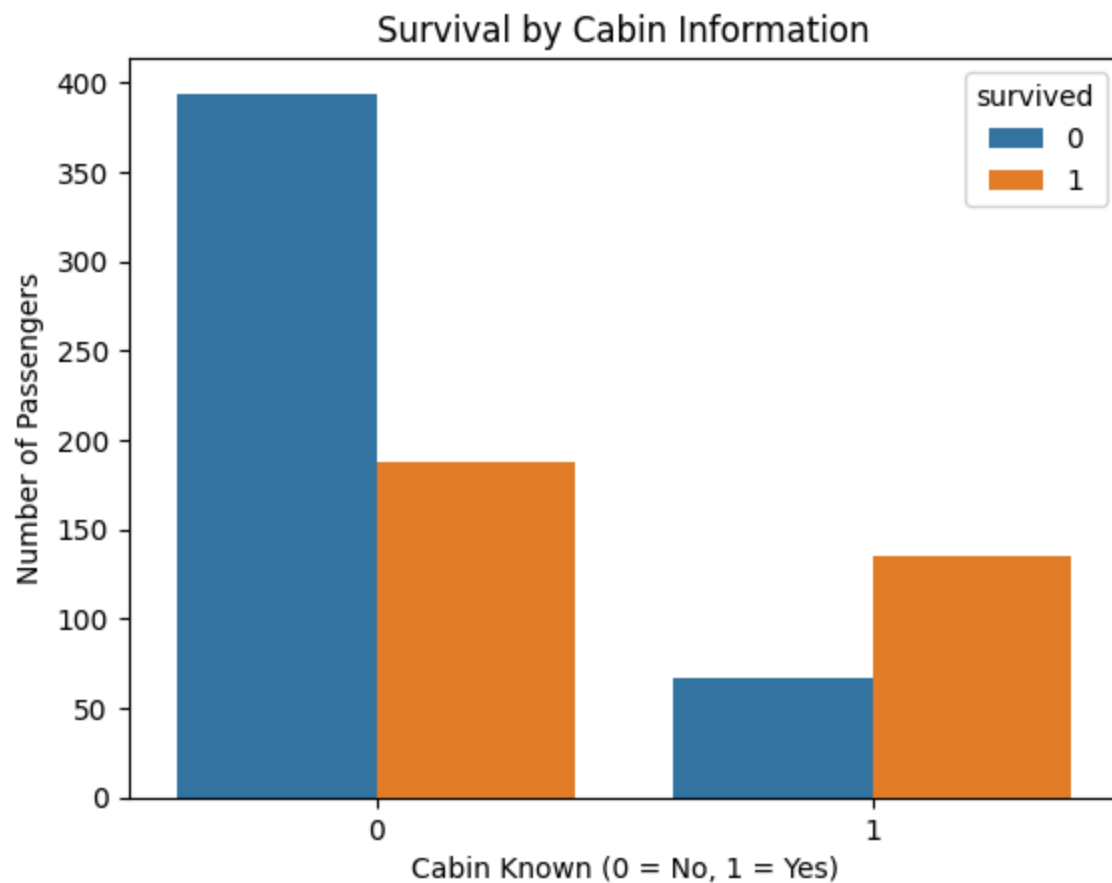

```

```
In [106... kashti["CabinKnown"].value_counts()
```

```
Out[106... CabinKnown  
0    582  
1    202  
Name: count, dtype: int64
```

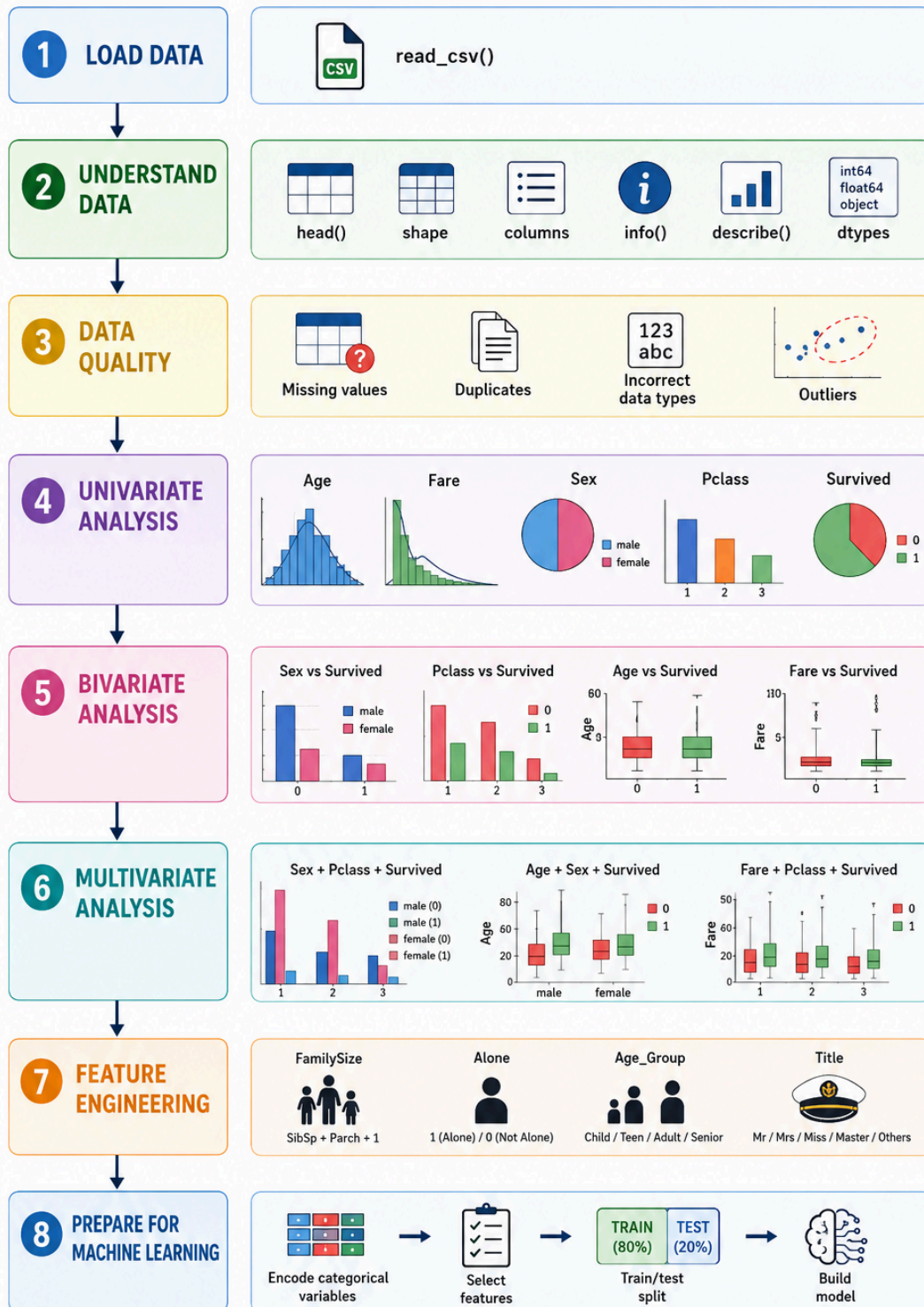
```
In [107... sns.countplot(  
    data=kashti,  
    x="CabinKnown",  
    hue="survived"  
)  
  
plt.title("Survival by Cabin Information")  
plt.xlabel("Cabin Known (0 = No, 1 = Yes)")
```

```
plt.ylabel("Number of Passengers")  
plt.show()
```



15. Final EDA checklist

TITANIC EDA PIPELINE



GOAL: DISCOVER INSIGHTS AND PREPARE DATA FOR PREDICTING SURVIVAL



TITANIC EDA PIPELINE WITH PYTHON CODES

1 LOAD DATA

- read_csv()

```
import pandas as pd
```

```
df = pd.read_csv('titanic.csv')
```

```
df.head() # quick peek
```

PassengerId	Survived	Pclass	Name	Sex	Age	Fare
0	1	0	Braund, Mr. Owen Harris	male	22	7.2500
1	2	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38	71.2833
2	3	1	Heikkinen, Miss. Laina	female	26	7.9250
3	4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	53.1000
4	5	0	Allen, Mr. William Henry	male	35	8.0500

2 UNDERSTAND DATA

- head()
- shape
- columns
- info()
- describe()
- dtypes

```
print("Shape:", df.shape)
print('Columns:\n', df.columns)
df.info()
df.describe(include='all').T
df.dtypes
```

Shape: (891, 12)

Columns:

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name',
       'Sex', 'Age', 'SibSp', 'Parch', 'Ticket',
       'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

Dtypes

PassengerId	int64
Survived	int64
Pclass	int64
Name	object
Sex	object
Age	float64
SibSp	int64
Parch	int64
Ticket	object
Fare	float64
Cabin	object
Embarked	object

dtype: object

3 DATA QUALITY

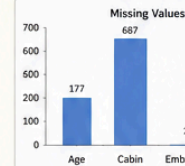
- Missing values
- Duplicates
- Incorrect data types
- Outliers

```
# Missing values
df.isnull().sum().sort_values(ascending=False)

# Duplicates
df.duplicated().sum()

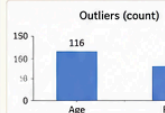
# Incorrect data types
df.dtypes

# Outliers (IQR method for Age & Fare)
Q1 = df[['Age', 'Fare']].quantile(0.25)
Q3 = df[['Age', 'Fare']].quantile(0.75)
IQR = Q3 - Q1
((df[['Age', 'Fare']] < (Q1 - 1.5*IQR)) |
 (df[['Age', 'Fare']] > (Q3 + 1.5*IQR))).sum()
```



Duplicates

Number of duplicates: 0



4 UNIVARIATE ANALYSIS

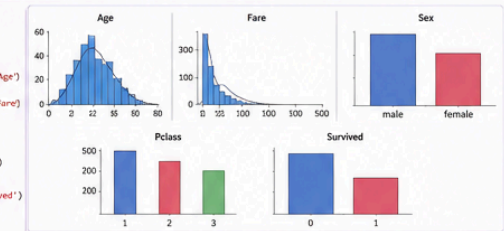
- Age
- Fare
- Sex
- Pclass
- Survived

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(15,10))

# Age
plt.subplot(2,3,1); sns.histplot(df['Age'], kde=True); plt.title('Age')
# Fare
plt.subplot(2,3,2); sns.histplot(df['Fare'], bins=30, kde=True); plt.title('Fare')
# Sex
plt.subplot(2,3,3); sns.countplot(x='Sex', data=df); plt.title('Sex')
# Pclass
plt.subplot(2,3,4); sns.countplot(x='Pclass', data=df); plt.title('Pclass')
# Survived
plt.subplot(2,3,5); sns.countplot(x='Survived', data=df); plt.title('Survived')

plt.tight_layout(); plt.show()
```



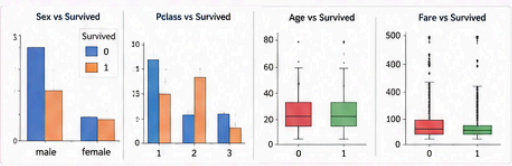
5 BIVARIATE ANALYSIS

- Sex vs Survived
- Pclass vs Survived
- Age vs Survived
- Fare vs Survived

```
plt.figure(figsize=(16,4))

# 1. Sex vs Survived
plt.subplot(1,4,1); sns.countplot(x='Sex', hue='Survived', data=df);
plt.title('Sex vs Survived')
# 2. Pclass vs Survived
plt.subplot(1,4,2); sns.countplot(x='Pclass', hue='Survived', data=df);
plt.title('Pclass vs Survived')
# 3. Age vs Survived
plt.subplot(1,4,3); sns.boxplot(x='Survived', y='Age', data=df);
plt.title('Age vs Survived')
# 4. Fare vs Survived
plt.subplot(1,4,4); sns.boxplot(x='Survived', y='Fare', data=df);
plt.title('Fare vs Survived')

plt.tight_layout(); plt.show()
```



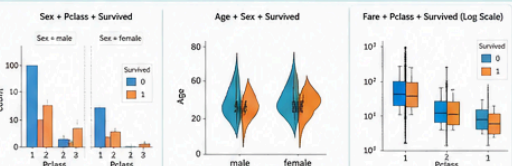
6 MULTIVARIATE ANALYSIS

- Sex + Pclass + Survived
- Age + Sex + Survived
- Fare + Pclass + Survived

```
# 1. Sex + Pclass + Survived
plt.figure(figsize=(15,4))
sns.catplot(x='Pclass', col='Sex', hue='Survived', kind='count',
            data=df, height=4, aspect=1);
plt.suptitle('Sex + Pclass + Survived', y=1.02); plt.show()

# 2. Age + Sex + Survived
plt.figure(figsize=(8,5))
sns.violinplot(x='Sex', y='Age', hue='Survived', data=df, split=True);
plt.title('Age + Sex + Survived'); plt.show()

# 3. Fare + Pclass + Survived
plt.figure(figsize=(8,5))
sns.boxplot(x='Pclass', y='Fare', hue='Survived', data=df);
plt.yscale('log'); plt.title('Fare + Pclass + Survived (Log Scale)');
plt.show()
```



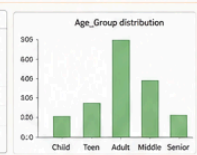
7 FEATURE ENGINEERING

- FamilySize
- Alone
- Age_Group
- Title

```
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df['Alone'] = (df['FamilySize'] == 1).astype(int)
df['Age_Group'] = pd.cut(df['Age'],
                          bins=[0,12,18,30,50,80],
                          labels=['Child', 'Teen', 'Adult', 'Middle', 'Senior'])

df['Title'] = df['Name'].str.extract(r'([A-Za-z-])\.', expand=False)
df['Title'] = df['Title'].replace(['Mlle', 'Ms', 'Miss',
                                   'Mrs', 'Mr', 'Major',
                                   'Capt', 'Don', 'Lady', 'Countess'],
                                   ['Teen', 'Adult', 'Middle', 'Senior'])
df[['FamilySize', 'Alone', 'Age_Group', 'Title']].head()
```

PassengerId	FamilySize	Alone	Age_Group	Title
0	2	0	Adult	Mr
1	1	1	Middle	Mrs
2	3	1	Adult	Miss
3	4	1	Middle	Mrs
4	5	1	Middle	Mr



8 PREPARE FOR MACHINE LEARNING

- Encode categorical variables
- Select features
- Train/test split
- Build model

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Select features
features = ['Pclass', 'Sex', 'Age_Group', 'FamilySize', 'Alone', 'Embarked', 'Title']
X = df[features]; y = df['Survived']

# Encode categorical variables
cat_cols = ['Sex', 'Embarked', 'Age_Group', 'Title']
X = pd.get_dummies(X, columns=cat_cols, drop_first=True)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)

# Build model
model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

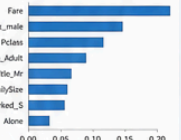
# Evaluate
print('Accuracy: ', round(accuracy_score(y_test, y_pred), 4))
print(classification_report(y_test, y_pred))
```

Model Performance

Accuracy: 0.8324

	precision	recall	f1-score	support
0	0.82	0.90	0.86	107
1	0.85	0.72	0.78	72
accuracy	0.84	0.81	0.83	179
macro avg	0.84	0.81	0.82	179
weighted avg	0.83	0.83	0.83	179

Feature Importance (Top 8)



🔑 This pipeline takes you from raw Titanic data to insights and a machine learning model for survival prediction!

The key EDA questions for Titanic

Think of EDA as asking the data questions:

"Kashti mein kon log zinda bachay?"

1. Did women survive more than men?
2. Did children survive more than adults?
3. Did 1st-class passengers survive more than 2nd/3rd class?
4. Did fare relate to survival?
5. Did traveling alone vs with family affect survival?
6. Did age + gender + class together explain survival better?
7. Which variables have the strongest relationship with Survived?